

BITÁCORA

Actividad de Evaluación 2 – Simulador de Procesos y Memoria

Paradigmas y Lenguajes de Programación II

Estudiante: Sophia Saquer – DNI 46857995

Carrera: Ingeniería en Sistemas de Información

Repositorio: <https://github.com/sophisaquer/Simulador-Procesos>

En esta bitácora registro, para cada encuentro de trabajo, las tareas que realicé, las decisiones que tomé, las dificultades que encontré y las pruebas que hice.

Encuentro 1 – 30/09/2026

Tareas realizadas

- Creé el repositorio personal Simulador-Procesos en GitHub.
- Leí la consigna y el Anexo I e identifiqué los requerimientos funcionales RF01 a RF10 y los casos mínimos de verificación.

Decisiones

- Elegí TypeScript como lenguaje y Vitest como framework de pruebas.
- Decidí organizar la solución como una biblioteca de clases, sin main ni menú de consola, y verificar todo mediante tests, como pide la consigna.
- Decidí trabajar con commits chicos, cada uno con una funcionalidad o con sus pruebas, para que el historial muestre el avance.

Encuentro 2 – 01/10/2026

Tareas realizadas

- Creé la estructura del proyecto: src/interfaces, src/modelos y tests, con la configuración de TypeScript y Vitest.
- Modelé BloqueMemoria e implementé Memoria, que inicia con un único bloque libre del tamaño total.
- Configuré el estado inicial del Simulador con memoria y quantum configurables.
- Modelé Proceso y EstadoProceso con los seis estados, y el registro y la consulta de procesos.
- Implementé la política FirstFit, la asignación de memoria con división de bloques y la liberación con coalescencia.
- Implementé la admisión de procesos al avanzar un tick y la cola de listos en PlanificadorRoundRobin.

Decisiones

- Separé las interfaces de las implementaciones en carpetas distintas.
- Puse la búsqueda del bloque en una clase aparte (FirstFit) que implementa IPoliticaAsignacion, para que Memoria no dependa del algoritmo concreto.
- Represento un bloque libre con PID null.

- La admisión recorre los procesos en orden de registro, y un proceso que no entra en memoria no impide admitir a los siguientes (RF03).
- La cola de listos tiene su propia clase, separada del Simulador.

Dificultades y correcciones

- TypeScript daba error al resolver los módulos con la configuración inicial. Lo resolví cambiando moduleResolution de "node" a "bundler" en tsconfig.json.
- La primera versión de BloqueMemoria quedó incompleta. La completé implementando la interfaz IBloqueMemoria.

Pruebas

- Bloques de memoria, memoria inicial y rechazo de tamaños inválidos.
- Configuración inicial del simulador, estados iniciales de los procesos y registro, con rechazo de PID duplicados, datos inválidos y procesos mayores que la memoria.
- FirstFit, asignación con división, asignación exacta y fallo sin modificar la memoria.
- Liberación y coalescencia, admisión, proceso en espera que no bloquea la admisión y orden FIFO de la cola.

Encuentro 3 – 02/10/2026 (madrugada)

Tareas realizadas

- Integré la cola al simulador: despacho y ejecución de un proceso por tick.
- Implementé la finalización de procesos con liberación de memoria en el mismo tick, y la rotación y renovación del quantum.
- Implementé la entrada y salida en Proceso y la integré al simulador.
- Definí y calculé las métricas: ocupación de memoria, utilización de CPU, cambios de contexto, memoria libre total, mayor bloque libre y fragmentación externa.
- Definí la vista del estado del sistema y la expuse desde el Simulador.
- Configuré GitHub Actions y completé el README.

Decisiones

- Cada tick sigue el orden de la consigna: admisión, actualización de bloqueados, despacho y ejecución, y actualización del reloj.
- La finalización tiene prioridad sobre el bloqueo, y el bloqueo sobre la rotación por quantum.
- Cuento un cambio de contexto solo por expulsión con otros procesos listos o por bloqueo de E/S, según la convención de RF09.
- Si la memoria libre total es 0, la fragmentación externa es 0%, para evitar la división por cero.
- Las consultas devuelven vistas de solo lectura (IVistaProceso, IEstadoSistema) en lugar de los objetos internos.

Dificultades y correcciones

- Al integrar la cola tuve que reorganizar avanzarTick(), y el despacho no quedaba en el lugar correcto del orden de fases. Lo corregí y agregué la prueba de ejecución por tick.
- Rehice las pruebas de coalescencia para separar los casos de fusión con el bloque de la izquierda y con el de la derecha.

- Al revisar el encapsulamiento noté que la memoria era un atributo público y que obtenerProcesos() devolvía los objetos Proceso, que se podían modificar desde afuera. Hice privado el atributo, reemplacé el retorno por vistas y adapté los tests del simulador.

Pruebas

- Secuencia completa de Round Robin del caso mínimo: P1, P1, P2, P2, P1 con un cambio de contexto.
- Renovación del quantum, finalización en el límite del quantum, bloqueo y retorno por E/S.
- Métricas: huecos de 100 y 300 con fragmentación del 25%, memoria llena y tick 0.
- Consulta del estado del sistema e invariantes: sin duplicados, sin solapamientos y admisión en el tick siguiente a una liberación.

Encuentro 4 – 02/10/2026 (mañana)

Tareas realizadas

- Validé los parámetros de asignación de memoria: PID y tamaño deben ser enteros positivos.
- Protegí las transiciones de los procesos: cada método verifica el estado de origen antes de cambiarlo.
- Ajusté la prueba de coalescencia con el bloque izquierdo.
- Protegí la cola del planificador: hacia afuera solo expone el orden de los PID con obtenerPids().

Decisiones

- Que Proceso lance un error ante una transición inválida, para que las reglas de estado estén en el propio objeto y no dispersas en otras clases.
- Que el planificador no entregue referencias a los procesos que almacena.

Dificultades y correcciones

- La prueba de fusión con el bloque izquierdo terminaba liberando toda la memoria, así que no comprobaba solo ese caso. Agregué un tercer proceso para que el bloque de la derecha quede ocupado y la fusión sea únicamente con la izquierda.

Pruebas

- Rechazo de transiciones de estado inválidas y de parámetros inválidos al asignar memoria.
- Consulta del orden de la cola sin exponer los procesos.
- El workflow de GitHub Actions pasó correctamente.

Encuentro 5 – 03/10/2026

Tareas realizadas

- Trabajé en el informe técnico.
- Redacté la introducción, el objetivo y el escenario, la descripción general de la solución y el diseño con las responsabilidades de cada clase.

- Redacté las decisiones de programación orientada a objetos, la sección de interfaces, abstracción, composición, encapsulamiento y SOLID, la política de memoria y los estados de los procesos con el orden de cada tick.
- Armé la matriz que relaciona RF01 a RF10 con las clases, los métodos y los tests.
- Redacté los resultados de las pruebas y la cobertura, y el análisis de fragmentación externa.

Decisiones

- Organicé el informe siguiendo los puntos que pide la consigna para la entrega individual.
- Justifiqué en el informe por qué no usé herencia ni clases abstractas y qué alternativa adopté: interfaces y composición.

Dificultades y correcciones

- No llegué a terminar el diagrama de clases, así que dejé la sección pendiente para completarla con el código final.
- La tabla de la matriz RF se desarmaba al pasar a PDF y el texto se salía de las columnas. Tuve que ajustar el ancho de las columnas.
- Las fórmulas de fragmentación no se mostraban bien en Word. Las rehice con el editor de ecuaciones.

Pruebas

- Verifiqué que los resultados del informe coincidieran con la última ejecución: 61 tests aprobados y 97,53% de cobertura de líneas.

Encuentro 6 – 04/10/2026

Tareas realizadas

- Revisé el trabajo contra la consigna y la rúbrica de evaluación.
- Agregué las políticas BestFit y WorstFit con sus pruebas.
- Agregué una prueba que ejecuta el mismo escenario de memoria con las tres políticas.
- Permití elegir la política al crear el Simulador.
- Actualicé el README, armé el diagrama de clases y corregí el diagrama de secuencia de admisión.
- Seguí trabajando en el informe técnico: lo actualicé con las tres políticas, agregué la sección de polimorfismo y la comparación de políticas, actualicé la matriz y los resultados, sumé las capturas de los tests, la cobertura y GitHub Actions, y agregué el encabezado en todas las páginas.

Decisiones

- Aunque la consigna pide al menos una política, agregué Best Fit y Worst Fit para demostrar el polimorfismo con implementaciones reales e intercambiables, ya que con una sola implementación no había comportamientos que intercambiar.
- Ante empates se elige el bloque de menor dirección; por eso la comparación es estricta y se queda con el primero encontrado.
- First Fit queda como política por defecto, para no cambiar el comportamiento ni romper las pruebas existentes.

- En el diagrama de admisión reemplacé FirstFit por la interfaz IPoliticaAsignacion, corregí el nombre asignarMemoria y agregué el loop sobre los procesos, para que coincida con el código.
- Dividí los cambios en commits de menos de 50 líneas.

Dificultades y correcciones

- Un commit no se registró porque no había agregado el archivo con git add. Lo resolví agregándolo y repitiendo el commit.
- El diagrama de clases es muy ancho y en el PDF del informe se ve chico. Dejé en el repositorio la versión en alta resolución y el archivo editable.

Pruebas

- 69 tests aprobados en 8 archivos (antes eran 61; actualicé el informe con los valores nuevos).
- Cobertura: 97,76% de sentencias, 95,03% de ramas, 100% de funciones y 97,71% de líneas.
- El workflow de GitHub Actions pasó correctamente.